马福泉 2333617 9

编译原理 4

4.1 (1) 消除左递归，改写文法为：

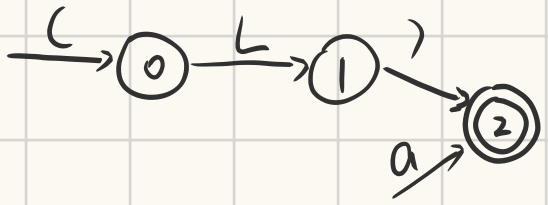
$$S \to (L) \mid a$$

$$L \to SL'$$

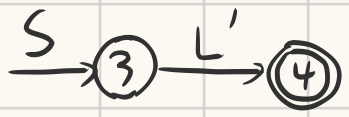
$$L' \to ,SL' \mid \varepsilon$$

(2) 转移图：

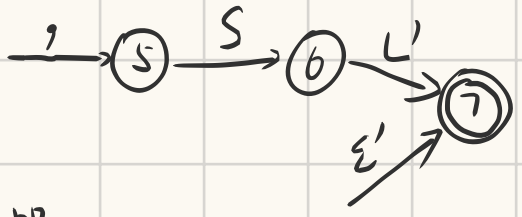
S:



L:



L':



(3) 预测分析器：

```
// S -> (L) | a
void S() {
    if (lookahead == '(') {
        match('(');
        L();
        match(')');
    } else if (lookahead == 'a') {
        match('a');
    } else {
        error();
    }
}
```

```
// L -> S L'
void L() {
    if (lookahead == '(' || lookahead == 'a') {
        S();
        L_prime();
    } else {
        error();
    }
}
// L' -> ,S L' | ε
void L_prime() {
    if (lookahead == ',') {
        match(',');
        S();
        L_prime();
    } else {;
        // ε 产生式：什么都不做，直接返回。
    }
}
```

(4) 输入 (a, (a, a)) 函数调用序列

① S()，识别"("，执行 match('(')，指针移到 a

② 进入 L()，看到 a，调用 S()

③ 内层 S() 看到 a，执行 match('a')，指针移到 ，内层 S() 结束

④ 回到 L()，调用 L_prime()

⑤ L_prime()看到 ， match(';') 指针移到 (，再次调用S()

⑥ 再进内层S()，看到 (，match('('), 指针移向a，调用L()

⑦ 进入内层L()，看到 a，调用S()，执行match(a')，指针移到) ，S()又结束(内层)

⑧ 回到内层L()，调用Lprime()，match(';')，调用S() match(a)，指针移到)

再调用Lprime()，看到)，走ε分支，结束

⑨ 逐层结束返回，最外层的S() match(')')，结束。

4.2

不可以，因为此文法不是LL(1)文法

不满足：当遇到同一个非终结符的多个产生式时

我们只看下一输入符号就有唯一决定

具体：S→ε

FIRST(aSbS) ={a}

FIRST(bSaS) ={b}

a和b又都在 FOLLOW(S)里面，不满足 LL(1)文法

4.3 ①计算FIRST(S)：

由 S→a。a是终结符，故a在FIRST(S)中。

由 S→SS+ 和 S→SS* ，FIRST取决于前面S，又目前S只能推导出a开头串

故 FIRST(S)={a}

②计算FOLLOW(S)：

首先结束标志$指定在其中

由 S→SS+，S后紧跟S，FIRST(S)={a}故a在FOLLOW(S)中，

S+中+为终结符，+也在FOLLOW(S)中

同理由S→SS*，*也在FOLLOW(S)中

综上 FOLLOW(S)={a,+,*,$}